Position, Trust, and the Algebra of Telemetry

From Ren Lin's Notebook, formatted and rewritten by Claude | Team 1280 EECS

Contents

1. Prerequsites 3
2. The Deceptively Simple Question 3
3. Poses Live in a Lie Group 3
4. Swerve Kinematics: Four Modules, One Body 4
5. The Problem with Certainty 4
6. Evidence: A Commutative Monoid 5
7. Three Gyroscopes and a Consensus 6
8. The Composite Trust Signal 7
9. The Estimator: Covariance as a Dial 7
10. Vision: The Hardware Stack 8
11. Vision: How It Plugs In 9
12. The System as a Whole 10

1. Prerequsites

simple category theory, group theory, and HOTT generally makes everything easier to understand. It's written and by me and checked with Claude to make the literature better and easier to understand.

2. The Deceptively Simple Question

Every match starts the same way: a 120-pound aluminum chassis is set down on a field, and a countdown begins. In those first seconds, the robot has no idea where it is. Not roughly. Not approximately. Literally nothing—a blank slate. Before it can score, before it can navigate, before it can do anything useful, it needs an answer to the oldest question in robotics: *where am I?*

The naive answer is "count the wheel rotations." If you know how far each wheel has turned and which way it was pointed, you can dead-reckon your position from a known starting point. And it works. Until a wheel spins in place during a collision. Until the gyroscope drifts. Until another robot shoves you sideways and the encoders report confident, pristine nonsense. The question is not just *how* to estimate position—it's how to know when to *trust* the estimate.

This document is an account of how this robot answers both questions at once.

3. Poses Live in a Lie Group

Before any algorithms, there is geometry. A robot moving on a flat field has three degrees of freedom: two translations (x, y) and one rotation θ. Together these form a **pose**, which is not merely a tuple of numbers—it is an element of the **special Euclidean group** $SE(2)$.

A pose can be represented as a 3×3 matrix:

$$p = \begin{pmatrix} \cos\theta & -\sin\theta & x \\ \sin\theta & \cos\theta & y \\ 0 & 0 & 1 \end{pmatrix} \in SE(2)$$

This representation makes composition natural. If the robot is at pose p_1 and moves to pose p_2 **relative to its own frame**, the new world pose is simply matrix multiplication $p_1 \cdot p_2$. The group axioms fall out automatically: there is an identity (the zero pose), every pose has an inverse (the "undo" transformation), and composition is associative. WPILib's `Pose2d` class carries exactly this structure, and the pose estimator maintains a running element of $SE(2)$ updated each control loop.

The **Lie algebra** $\mathfrak{se}(2)$ is the tangent space at the identity—it describes instantaneous velocities rather than positions. An element $\xi = (v_x, v_y, \omega)$ is precisely a `ChassisSpeeds` object: forward velocity, strafe velocity, and angular velocity. The exponential map $\exp$:

$\mathfrak{se}(2) \to SE(2)$ converts a constant twist applied for time Δt into the resulting pose displacement:

$$p_{t+\Delta t} \approx p_t \cdot \exp(\xi \cdot \Delta t)$$

For small Δt, this is the familiar Euler integration step, but the group-theoretic framing clarifies why you must compose (multiply) rather than add: poses are not vectors, and naive coordinate addition ignores the rotational coupling.

4. Swerve Kinematics: Four Modules, One Body

A swerve drivetrain has four independently steered and driven wheels. Each module i sits at position $\boldsymbol{r}_i = \big(r_{ix}, r_{iy}\big)$ relative to the robot center and can point its wheel in any direction φ_i. Given a target chassis twist $\xi = \big(v_x, v_y, \omega\big)$, the velocity each module must produce is:

$$\boldsymbol{v}_i = \begin{pmatrix} 1 & 0 & -r_{iy} \\ 0 & 1 & r_{ix} \end{pmatrix} \begin{pmatrix} v_x \\ v_y \\ \omega \end{pmatrix}$$

This is the **forward kinematics map**: a linear function from the three-dimensional twist space to the two-dimensional velocity space of each module. Stacking all four modules gives an 8×3 Jacobian A, and the full system is:

$$\boldsymbol{b} = A\xi$$

where $\boldsymbol{b} \in \mathbb{R}^8$ holds the eight measured wheel velocity components. Running this **backward**—from measured wheel velocities to estimated twist—is the core of encoder odometry, solved in the least-squares sense:

$$\hat{\xi} = \left(A^\top A\right)^{-1} A^\top \boldsymbol{b}$$

The SwerveDriveKinematics class in WPILib solves exactly this system, wrapping the pseudoinverse in an efficient form. The integrated pose is then updated by composing $\exp\big(\hat{\xi} \cdot \Delta t\big)$ onto the current estimate at up to 250 Hz on a dedicated odometry thread, decoupled from the 50 Hz robot loop via a concurrent queue architecture.

5. The Problem with Certainty

Dead-reckoning from wheel encoders is fast, smooth, and wrong in interesting ways. A wheel lifted off the ground still reports rotation. A robot shoved sideways by a defender accumulates position error silently. A collision bounces the gyroscope reading for a fraction of a second—long enough to corrupt dozens of integration steps.

The tempting response is to also use a camera: mount it on the robot, detect AprilTag fiducial markers placed around the field, and compute an absolute pose from their known positions. This vision pipeline is accurate and drift-free, but it is also noisy, ambiguous, and sometimes just wrong. A single tag seen at a glancing angle can produce two plausible pose solutions. A distant tag produces a noisier estimate than a close one.

Neither source is better than the other. They are complementary, and what the system needs is a principled way to decide, moment by moment, how much to trust each one.

6. Evidence: A Commutative Monoid

Trust is modeled as **evidence**—a value in $[0, 1]$ where 1 means "no reason to doubt" and 0 means "completely disbelieved." The key design insight is that evidence values compose multiplicatively:

$$e_1 \otimes e_2 = e_1 \cdot e_2$$

This makes $([0, 1], \otimes, 1)$ a **commutative monoid**: the operation is associative and commutative, and 1 is the unit (no evidence reduces to full trust by default). Composing evidence from multiple independent checks yields a value that is no larger than any individual check—the intersection of all reasons to trust.

A **sensor disagreement** $\varepsilon \geq 0$ (an absolute error between two sources) is lifted into evidence space by a **Gaussian morphism**:

$$\mu_\sigma : \mathbb{R}_{\geq 0} \to [0, 1], \quad \mu_\sigma(\varepsilon) = e^{-\varepsilon/\sigma}$$

The parameter σ is the characteristic scale: disagreements much smaller than σ produce evidence near 1; disagreements much larger than σ collapse evidence toward 0. Crucially, μ_σ is itself a monoid morphism from $(\mathbb{R}_{\geq 0}, +, 0)$ to $([0, 1], \otimes, 1)$, since:

$$\mu_\sigma(\varepsilon_1 + \varepsilon_2) = e^{-(\varepsilon_1+\varepsilon_2)/\sigma} = \mu_{\sigma(\varepsilon_1)} \otimes \mu_{\sigma(\varepsilon_2)}$$

Additive errors in sensor space translate directly into multiplicative reductions in trust space. This is not an arbitrary design choice—it is the statement that the diagram below **commutes**: it does not matter whether you add the errors and then lift, or lift each error individually and then compose.

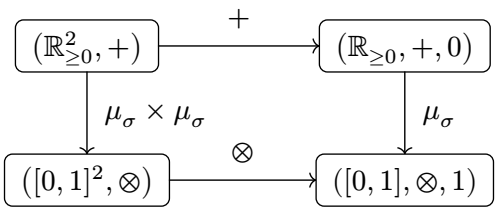

Figure 1: The Gaussian morphism μ_σ as a monoid homomorphism. Both paths from the top-left to the bottom-right agree: $\mu_{\sigma(\varepsilon_1+\varepsilon_2)} = \mu_{\sigma(\varepsilon_1)} \otimes \mu_{\sigma(\varepsilon_2)}$. The monoid unit is preserved as well: $\mu_{\sigma(0)} = e^0 = 1$.

7. Three Gyroscopes and a Consensus

The robot carries three independent measurements of angular velocity ω:

- G_1: the Pigeon2 IMU on the CANivore bus (Phoenix 6, hardware time-synchronized)
- G_2: the NavX2 connected via MXP UART (three-axis MEMS gyroscope)
- G_3: the kinematic omega derived from swerve module velocities via $\hat{\xi}$

No single source is authoritative. Instead, pairwise evidence is computed for each pair:

$$e_{12} = \mu_\sigma(|G_1 - G_2|), \quad e_{13} = \mu_\sigma(|G_1 - G_3|), \quad e_{23} = \mu_\sigma(|G_2 - G_3|)$$

The topology of this agreement network is a triangle: each source is connected to both others by an evidence edge, and a source's **credibility** is the product of the two edges incident to it—a source is trusted when it agrees with both partners simultaneously.

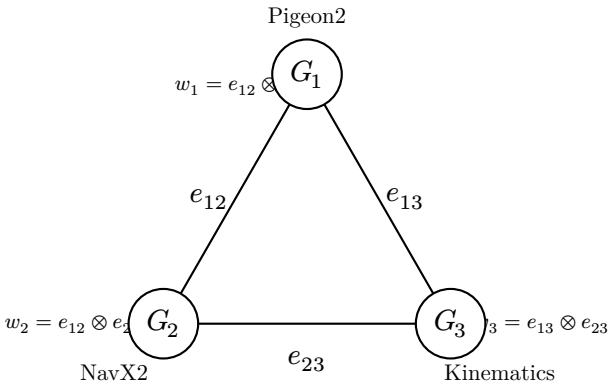

Figure 2: Pairwise agreement triangle for the three angular velocity sources. Each edge carries an evidence value; each node's weight is the product of its two incident edges. A source is discredited only when at least one of its neighbors disagrees with it.

Each source is then weighted by how well it agrees with *both* others:

$$w_1 = e_{12} \otimes e_{13}, \quad w_2 = e_{12} \otimes e_{23}, \quad w_3 = e_{13} \otimes e_{23}$$

The **consensus angular velocity** is the trust-weighted mean:

$$\hat{\omega} = \frac{w_1 G_1 + w_2 G_2 + w_3 G_3}{w_1 + w_2 + w_3}$$

If all three sources badly disagree—$w_1 + w_2 + w_3 < 10^{-9}$—the system falls back to G_1 (the Pigeon2), the most hardware-accurate source. The **overall gyro consensus evidence** is:

$$e_{\text{gyro}} = e_{12} \otimes e_{13} \otimes e_{23}$$

a single number summarizing whether the three rotation sensors are telling a consistent story.

8. The Composite Trust Signal

Gyro consensus is one of several checks that run each control loop. The full set of evidence sources is:

Wheel slip $e_{\text{slip}} = \mu_{\sigma_{\text{slip}}}(|\omega_{\text{odo}} - \hat{\omega}|)$, where $\omega_{\text{odo}} = \Delta\theta/\Delta t$ from the pose delta. When wheel encoders report a rotation rate that disagrees with the inertial consensus, at least one module has lost contact with the ground.

Angular jerk $e_{\text{jerk}} = \mu_{\sigma_{\text{jerk}}}\left(|\dot{\hat{\omega}}|\right)$. A sudden spike in angular velocity indicates a collision or tip event. The jerk is computed as the finite difference of $\hat{\omega}$ across loop iterations.

Linear bump acceleration $e_{\text{bump}} = \mu_{\sigma_{\text{bump}}}(\|\boldsymbol{a}_{\text{world}}\|)$, where $\boldsymbol{a}_{\text{world}}$ is the world-linear acceleration from the NavX2 (converted from g to m/s^2). Large accelerations in the plane indicate an impact.

Input correlation When the commanded chassis speeds are near zero but the robot is moving, something external is pushing it (defensive play). When commanded and actual velocities point in opposite directions, the odometry is being driven by a force the control system did not produce.

Drive motor CAN faults If all four TalonFX motors report a supply current of -1 A —the Phoenix sentinel value for a disconnected CAN device—the drivetrain is electrically faulted and encoder data is meaningless.

Because $\otimes$ is associative, these sources chain into a single composite value in any order; the code applies them left-to-right, which the diagram below makes concrete.

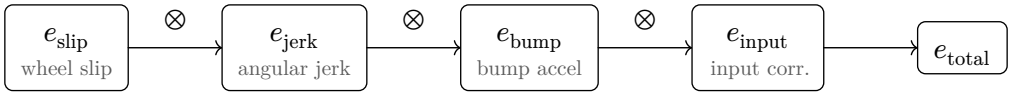

Figure 3: Sequential monoid composition producing the composite odometry trust. Because $\otimes$ is associative and commutative, the order of composition does not affect the result—only which failure modes are active. A CAN fault short-circuits this chain by forcing $e_{\text{total}} = 0$ regardless.

These sources compose into a single **composite odometry trust**:

$$e_{\text{total}} = e_{\text{slip}} \otimes e_{\text{jerk}} \otimes e_{\text{bump}} \otimes e_{\text{input}}$$

with the convention that a CAN fault forces $e_{\text{total}} = 0$. This scalar, cached as `cachedOdometryTrust`, gates every subsequent decision about how much to believe the wheel encoders.

9. The Estimator: Covariance as a Dial

WPILib's `SwerveDrivePoseEstimator` maintains a Kalman-filter-like state: a best-guess pose $\hat{p} \in SE(2)$ and a 3×3 covariance matrix Σ representing uncertainty in (x, y, θ).

Each vision measurement arrives with its own declared standard deviations—the estimator fuses it with the running state proportionally to the relative confidences.

The trust system controls the covariance passed with each vision measurement. Two matrices bracket the range:

$$\Sigma_{\text{best}} = \text{diag}(0.05^2, 0.05^2, 0.04^2), \quad \Sigma_{\text{worst}} = \text{diag}(0.60^2, 0.60^2, 0.40^2)$$

The effective standard deviations are linearly interpolated by a combined trust factor:

$$\alpha = 0.2 + 0.8 \cdot \left(e_{\text{total}} \cdot e_{\text{target}}\right), \quad \alpha \in [0.2, 1.0]$$

$$\Sigma_{\text{vision}(\alpha)} = (1 - \alpha)\Sigma_{\text{worst}} + \alpha\Sigma_{\text{best}}$$

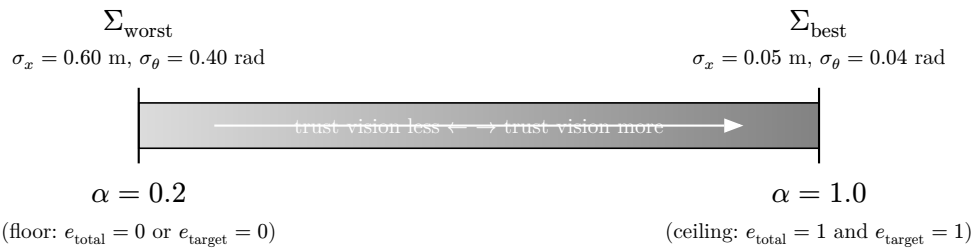

Figure 4: The covariance interpolation dial. As composite odometry trust e_{total} and per-measurement target evidence e_{target} both approach 1, the vision covariance shrinks toward Σ_{best} and the Kalman filter weights vision measurements more heavily. The floor at $\alpha = 0.2$ prevents the filter from ignoring vision entirely even during a full odometry failure.

The 0.2 floor prevents the estimator from completely ignoring vision even when odometry is degraded. The per-measurement factor e_{target} accounts for AprilTag geometry:

$$e_{\text{target}} = \begin{cases} 1 & \text{if } n_{\text{tags}} \geq 2 \\ 1 - a/a_{\max} & \text{if } n_{\text{tags}} = 1 \end{cases}$$

where $a \in [0, 1]$ is the **pose ambiguity**—the ratio of the second-best to best PnP reprojection error. A value near 1 means two pose solutions are nearly indistinguishable, so single-tag confidence is low. A value near 0 means the tag geometry is clear.

10. Vision: The Hardware Stack

Our hardware stack is in a superpoised state of simplicity and uncertainty, we adopted PhotonVision at the end of 2024 after horrible experiences with the Limelights, and it has proven to be as accurate if not more than it's closed hardware counterparts. Hence it's simplicity; however unlike most teams (unless you just happen to be *Mechanical Advantage*) putting a $400 Mac Mini on the robot is unheard of – most teams opt for the cheap and accessable single board aarch64 computers commonly used for vision/neural

processing purposes. The original plan was to hook up the cameras to a cv object detection model ontop of our rudimentary localization software, however having two of those cheap single board computers isn't going to cut it. To maxmize our processing utility we've decided on the most powerful aarch64 CPU on the market, the Mac M series chips. We stripped the computer down until it's just it's mainboard and the lights, added a simple 75w DC-DC converter that filtered the dirty 12V+ incoming power in a more modulated 11.9 volts which the mac mini happily took. While testing and ensuring the feasibility(i.e. compiling the Asahi Kernel on nixos) the processor never drew more than 5 amps of current further solidifying the electrical feasibility of this co-processor. Having daily drove NixOS on an MacBook Pro M1 (2021) our original plan was to use NixOS to run PhotonVision, but after weeks of testing and debugging something seems to be bothering the MrCal tool of PhotonVision which hindered the 3D tracking needed for odometry/vision measurements to be accurate. So a switch to Fedora 42(Asahi Remix) was needed. Now, you may ask *why not MacOS?* to that I say, systemd is way easier to manage in our enviornment, none of our programming leads have had plesant experiences on MacOS as daily drivers let alone a server, furthermore, I don't really know if Aqua(the DE) would be still active while running in a headless state(which would increase the power draw from our strict power diet) so a decision was made to make sure that it works on a Linux operating system.

11. Vision: How It Plugs In

Perspective-n-Point (PnP) pose estimation. AprilTag fiducial markers are printed at known positions on the field, encoded in an `AprilTagFieldLayout` (the 2026 Reefscape Welded field). Each tag corner has a known 3D world coordinate P_i. The camera observes 2D image projections $\boldsymbol{p}_i$ of those corners. Given the camera intrinsic matrix K (focal length, principal point), PnP solves for the camera pose $T \in SE(3)$ minimizing total reprojection error:

$$T^* = \arg\min_T \sum_i \|\boldsymbol{p}_i - \pi(T \cdot P_i)\|^2$$

where π is the pinhole projection function. With a single tag (four corners, four equations), the system is solvable but can admit two solutions rotated about the tag plane—this is "tag flipping," measured by the ambiguity ratio. With two or more tags the system is overdetermined and the solution is globally unique: multi-tag PnP.

PhotonVision and the Camera abstraction. Each physical camera runs a PhotonVision pipeline on a coprocessor. The `PhotonCamera` Java object in the robot code polls the pipeline results over NetworkTables. A `PhotonPoseEstimator` wraps each camera with its robot-to-camera transform (a `Transform3d` encoding the exact mount geometry) and converts `EstimatedRobotPose` results into the robot's world frame.

Each `Camera.update()` call returns a list of `VisionMeasurement` records:

```
record VisionMeasurement(
    Pose2d pose, double timestampSeconds,
    double distanceMeters, double ambiguity, int numTargets)
```

These records carry everything the trust pipeline needs. The distance gates out measurements from tags that are too close (unstable angle) or too far (noisy pixels). The timestamp is used to detect high-latency measurements that arrived after the robot's pose has drifted significantly.

The injection interface. `VisionSubsystem` holds a `VisionMeasurementConsumer` functional interface—a callback injected at construction time pointing to `OdometryDrivetrain::addVisionMeasurement`. Each accepted measurement travels through range filtering, latency filtering, and trust-aware covariance interpolation before landing in the Kalman estimator. Nothing bypasses the trust pipeline: the base-class `addVisionMeasurement(Pose2d, double)` overload is intentionally not used anywhere in the vision path, preventing accidental injection of unvetted measurements.

Two cameras currently run the pipeline (front and back), providing overlapping field coverage. An auxiliary camera transform (`AUX_CAMERA_TRANSFORM`) is defined for a potential third camera; wiring it into the subsystem is a one-line addition to the `cameras` array.

12. The System as a Whole

What makes this design satisfying is that the trust architecture is not bolted on top of a naive estimator—it is part of the same mathematical object. Evidence is a monoid, and monoids compose freely. Adding a new failure mode means defining a new Gaussian morphism and composing it with `and()`. Removing a check is equally clean. The algebra enforces a contract: every evidence value lives in $[0, 1]$, every composition can only reduce trust, and the unit 1 means "nothing suspicious detected."

The Kalman estimator sits at the boundary between the algebraic trust world and the probabilistic estimation world. Evidence in $[0, 1]$ is mapped to covariance matrices through the linear interpolation, which is admittedly a heuristic bridge—not a derived statistical relationship. But it is a principled heuristic: monotone, bounded, and tunable by adjusting two diagonal matrices.

The robot, by the start of autonomous, knows where it is.